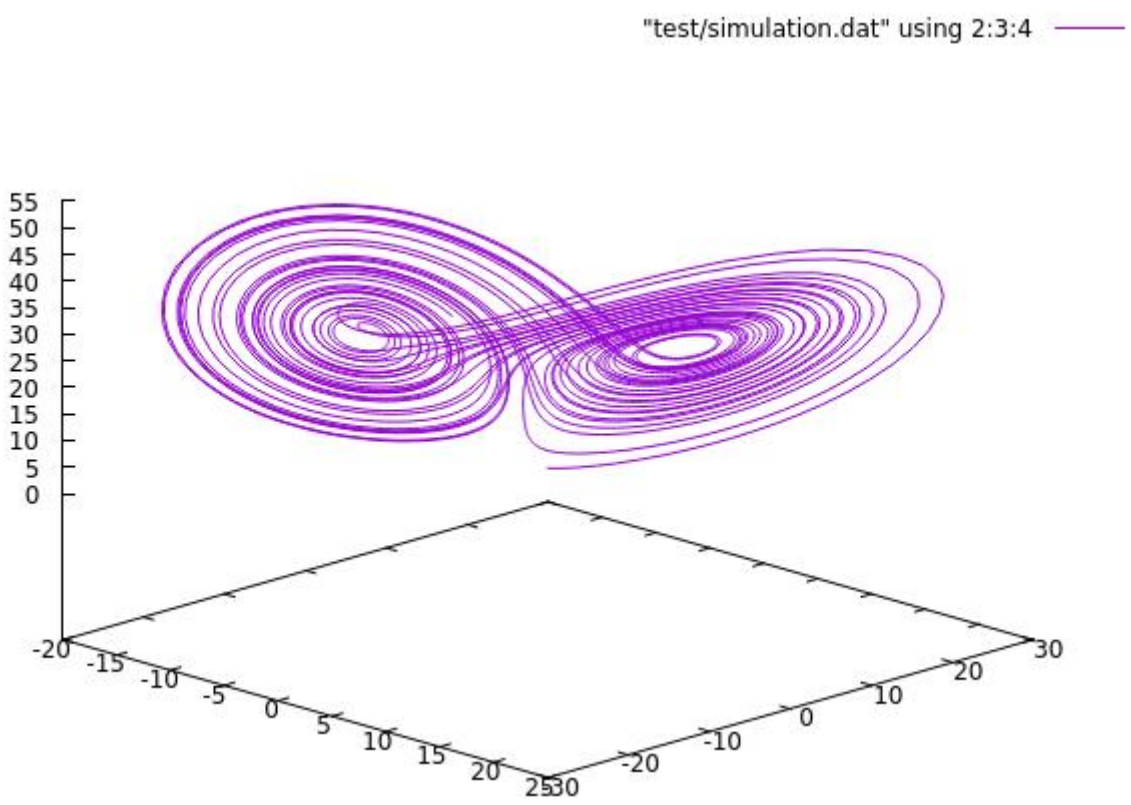
Projet 2025 - Informatique Appliqué 2

Attracteur de Lorenz,
Oscillateur de Van de Pol et Harmonique



Par Grelet Thomas et Dejean Louis

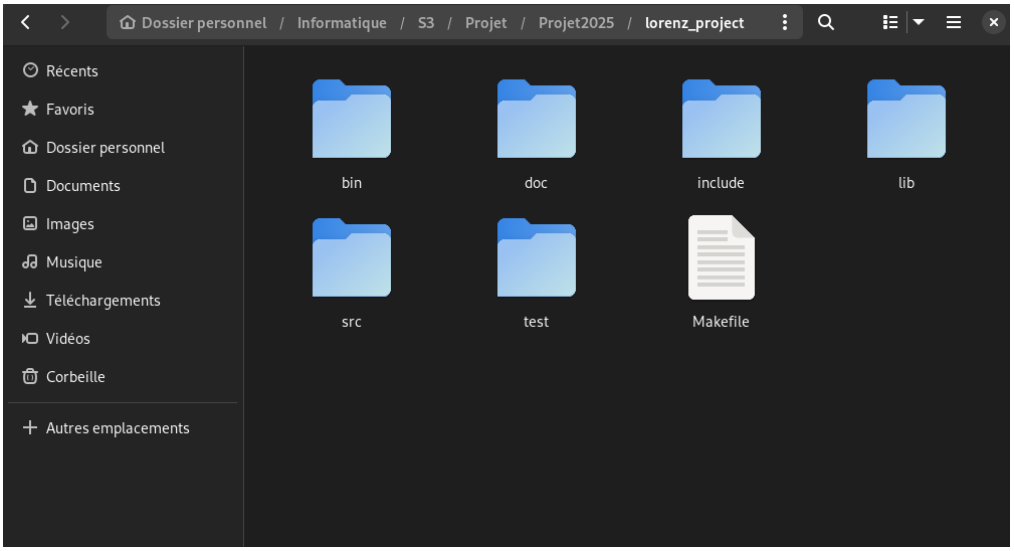
Introduction

Le projet consiste à modéliser et représenter la trajectoire d'un point soumis à des forces dynamiques, en utilisant des équations différentielles. Les équations de Lorenz, bien connues pour leur rôle dans l'étude du chaos, serviront de modèle principal. Ce système de trois équations différentielles couple la position (x,y,z) d'un point à sa vitesse dans un espace tridimensionnel. Les oscillateurs de Van der Pol et Harmonique viennent s'ajouter aux choix de l'utilisateur. En intégrant ces équations pas à pas avec un incrément de temps dt, nous calculons la trajectoire complète du point sur une période de temps donnée.

Ce projet mêle théorie des systèmes dynamiques, implémentation en langage C, et visualisation graphique avec l'outil *Gnuplot*.

I. Organisation des fichiers

La première étape a été de créé un dossier privé et partagé sur *github*, site sur lequel on peut travailler en collaboration et mettre au point nos fichiers, en mettant en collaborateurs les deux membres du groupe ainsi que le professeur attitré. Ensuite on a organisé notre dossier en plusieurs sous-dossiers pour bien ranger les fichiers.



- bin : dossier contenant les fichiers exécutables
- doc : dossier contenant le fichier texte README.md ainsi que ce fichier pdf
- include : dossier contenant tous les fichiers .h (déclarations)
- lib : dossier contenant tous les fichiers .o (sorties)
- src : dossier contenant tous les fichiers .c (sources)
- test : dossier contenant tous les fichiers .dat (données)

Enfin le fichier *Makefile* qui sert a automatisé les liaisons et compilations de fichiers.

II. Fonctions du projet

Pour saisir l'essence même du projet il faut comprendre ces fonctions et force est de dire que parfois elles sont assez coriaces l'exemple même étant la fonction main, ici nous expliquerons comment marche les différentes fonctions du projet mais aussi à quoi elles servent dans le but de bien comprendre comment fonctionne cette ensemble complexe :

1) La fonction **initialise_lorenz** :

Cette fonction prend en paramètres un pointeur vers une structure LorenzParams où sont stockées les constantes nécessaires aux équations de Lorenz, Cette fonction affecte donc des valeurs aux constantes se situant dans la structure LorenzParams, ces valeurs (constantes) sont donc les paramètres de nos équations de Lorenz. L'importance de cette fonction est qu'elle définie le comportement du système sans cela le comportement du système ne pourrait pas être simulé. Dans le programme cette fonction est appelée pour initialiser un modèle de Lorenz.

A noter que initialise_oscillator et initialise_van_der_pol fonctionnent exactement pareil seul le nombre de paramètres change.

2) La fonction **actualise_lorenz** :

Cette fonction calcule les dérivées des variables x, y et z à l'instant actuel en appliquant les équations différentielles, ces dérivées représentent les vitesses de changement des variables x, y et z au fil du temps, les nouvelles valeurs des variables sont calculées en intégrant les dérivées, Cette mise à jour est effectuée par l'addition des dérivées multipliées par le temps dt, ce qui simule l'évolution des variables sur un intervalle de temps. La fonction actualise_lorenz permet de calculer l'évolution du système dynamique basé sur les équations de Lorenz à chaque étape de la simulation. La fonctionne est appelée dans la boucle principale de simulation pour mettre à jour les variables à chaque itération.

A noter que actualise_oscillator et actualise_van_der_pol fonctionnent exactement de la même manière.

3) La fonction **main** :

La fonction main est le point d'entrée principal d'un programme C. Elle est exécutée au démarrage du programme et coordonne l'ensemble des étapes de la simulation du système dynamique (Lorenz, Van der Pol, Oscillateur). Voici une explication détaillé de son fonctionnement :

a) Allocation mémoire pour les paramètres des systèmes dynamiques : La mémoire est allouée pour les structures qui stockeront les paramètres spécifiques à chaque système dynamique (Lorenz, Van der Pol, Oscillateur). Cela permet de stocker les paramètres du modèle pour chaque système.

b) Vérification de l'allocation mémoire : Si l'allocation échoue, un message d'erreur est affiché, et le programme se termine prématurément.

c) Initialisation des systèmes dynamiques dans un tableau : Un tableau de 3 éléments est alloué pour stocker les trois systèmes dynamiques : Lorenz, Van der Pol, Oscillateur.

d) Assignation des propriétés à chaque système dynamique : Chaque élément du tableau « systems » représente un système dynamique. Pour chaque système on a le nom name, puis une fonction pour initialiser « init_params » , une fonction pour mettre à jour « update », un pointeur vers les paramètres du système « params ».

e) Choix du système par l'utilisateur : L'utilisateur est invité à choisir un système dynamique parmi les 3 systèmes disponibles. La sélection est lue et stockée dans la variable choice.

f) Validation du choix de l'utilisateur ; Si l'utilisateur fait un choix invalide, un message d'erreur est affiché et la mémoire allouée est libérée avant de quitter le programme.

g) Initialisation du système choisi : Le système dynamique sélectionné est initialisé en appelant la fonction appropriée (init_params), qui configure les paramètres de ce système.

h) Demande des paramètres de simulation de l'utilisateur : L'utilisateur est invité à entrer les conditions initiales pour la simulation, notamment la position initiale des variables d'état x,y,z ainsi que l'incrément de temps dt et la durée t_max.

i) Calcul du nombre d'étapes de simulation et ouverture du ficher de sortie : Le nombre d'étapes de simulation est calculé en fonction du temps maximal et de l'incrément de temps. Un ficher de données (simulation.dat) est ouvert pour enregistrer les résultats de la simulation.

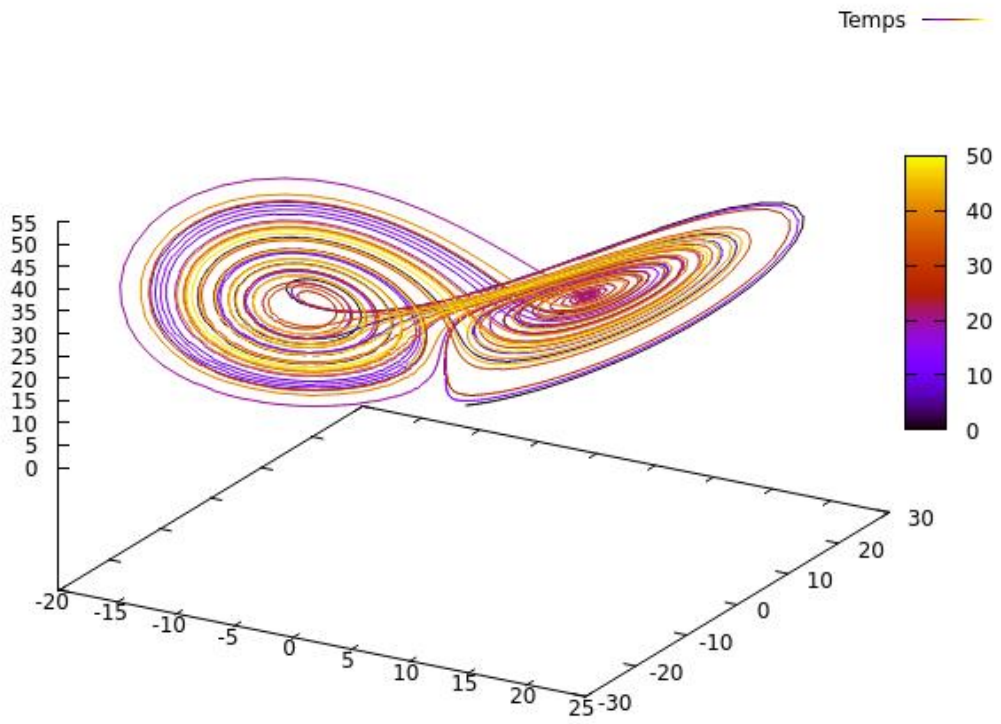
j) Simulation et mise à jour du système dynamique : A chaque itération la fonction update du système choisi est appelée pour mettre à jour les variables d'état x, y, z. Les résultats sont enregistrés dans le ficher de données, incluant la position les vitesses et la vitesse totale.

k) Fermeture du ficher et libération de la mémoire : Le ficher de données est fermé et la mémoire allouée pour les paramètres et les systèmes est libérée.

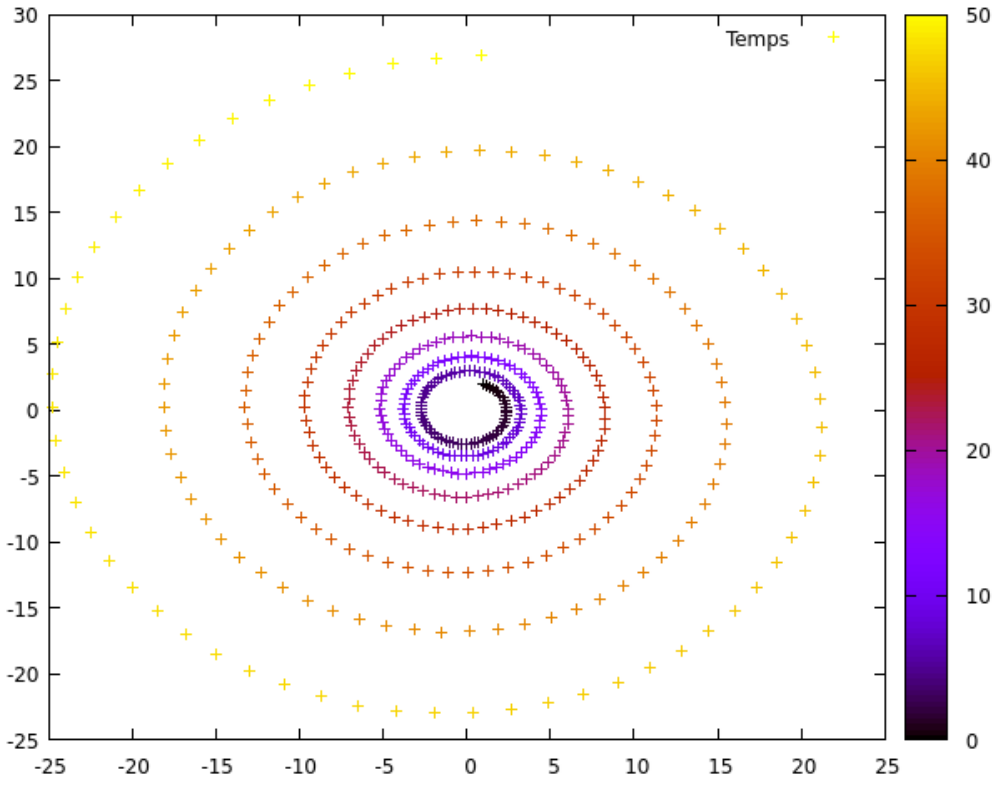
Chaque étape sera précisé sur le code.

Conclusion

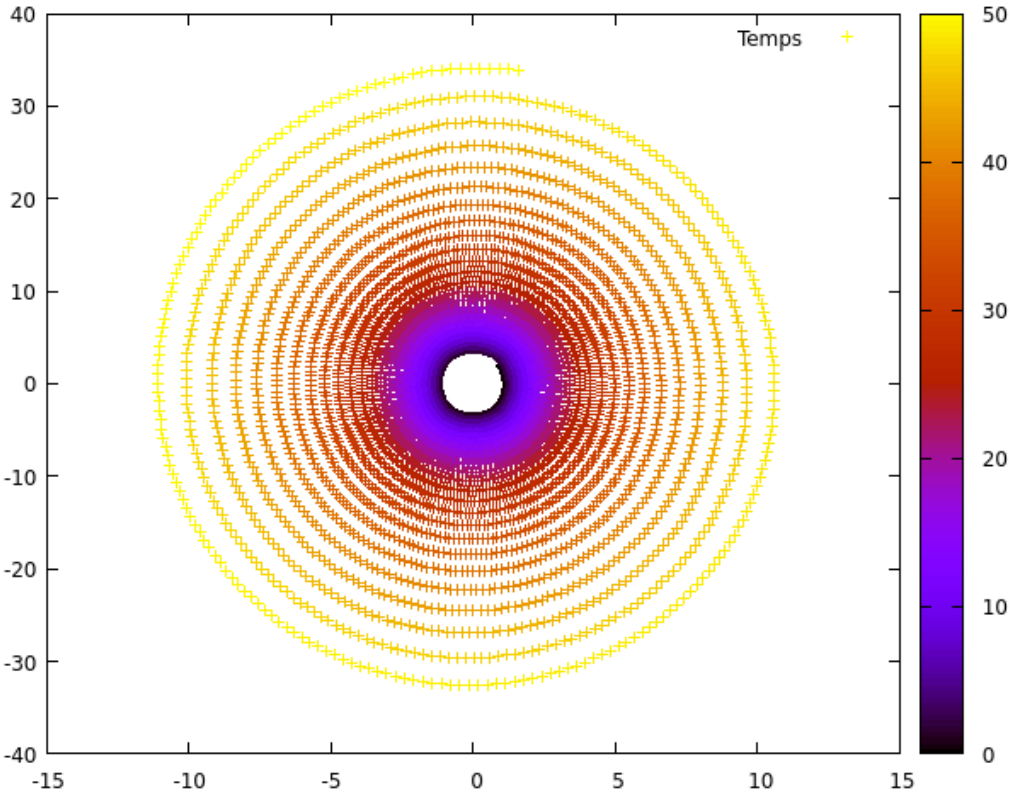
Nous pouvons donc tracer ce qu'on appelle un attracteur de Lorenz, grâce aux équations de Lorenz, ainsi que les Oscillateurs de Van der Pol et Harmonique. Voici quelques exemples :



Attracteur de Lorenz



Oscillateur de Van der Pol



Oscillateur Harmonique